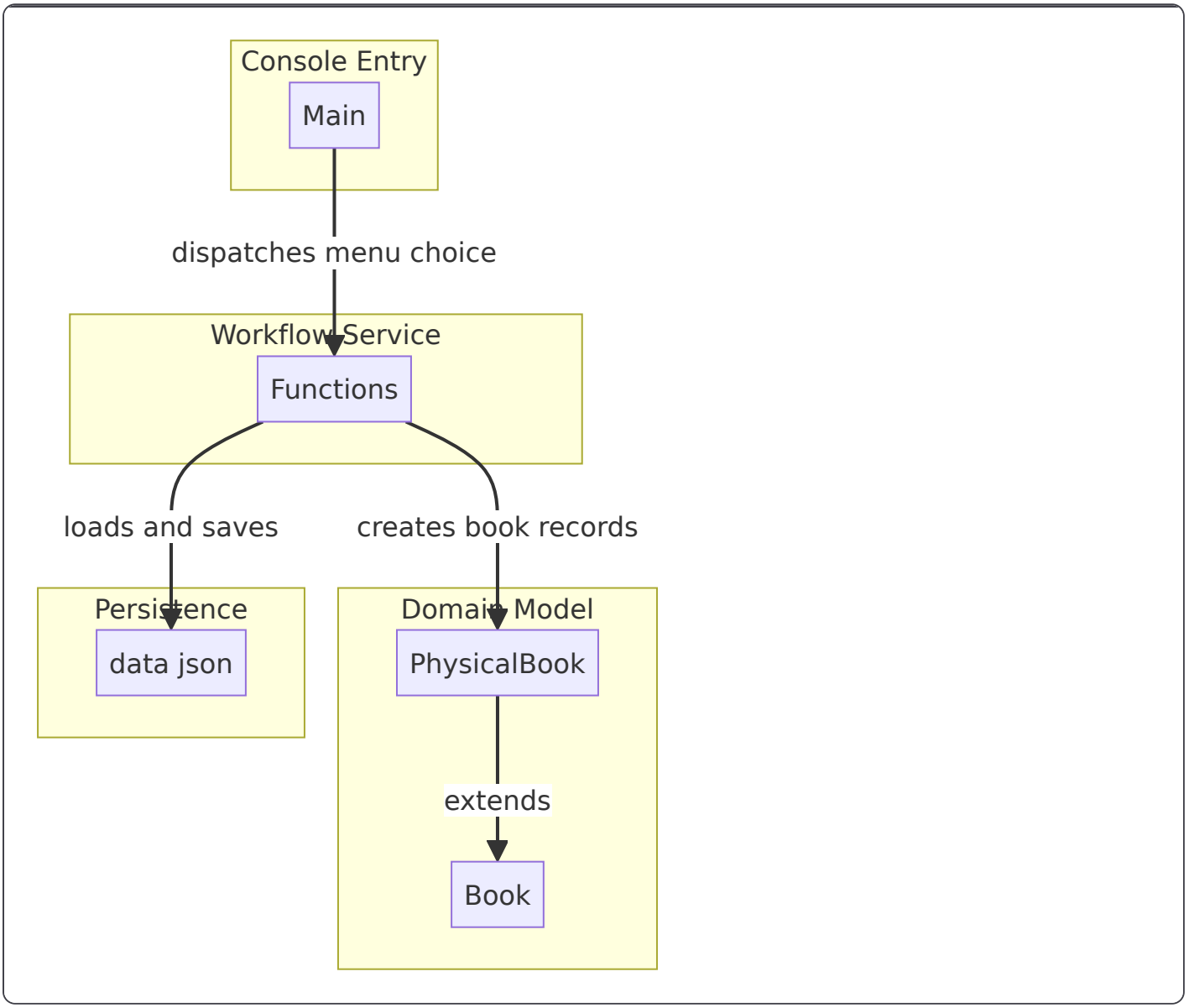
Book Management Domain - Interactive Book Lifecycle Workflows

Overview

This feature is the repository's core inventory service boundary. It lets a console operator add books, review the full catalog, check a book's availability by ISBN, toggle checkout state, and permanently remove a record from the local JSON store.

The workflow is centered on `src/ Functions.java`, which owns the in-memory inventory state, loads and saves `data.json`, and performs all ISBN-based record operations. `src/ Main.java` only dispatches menu choices into those workflows, while `src/ Book.java` and `src/ PhysicalBook.java` define the domain shape that is serialized into JSON.

Architecture Overview



Component Structure

Main.java

NOTE

> `Main.main` closes its `Scanner` inside the `while (run)` loop, which shuts down `System.in` after the first menu action. Because `Main` and `Functions` both read from the same console stream, the first `scanner.nextLine()` in `Functions.addBook()` can also consume the newline left behind by `scanner.nextInt()`.

src/Main.java

`Main` is the console entry point. It prints the library menu, reads the operator's selection with `scanner.nextInt()`, and dispatches directly into the `Functions` workflows.

Runtime variables inside `main`:

Name	Type	Purpose
`run`	`boolean`	Controls the menu loop
`scanner`	`Scanner`	Reads the menu option from standard input
`func`	`Functions`	Invokes the book lifecycle workflows

Console menu text:

```
--- Library Menu ---
1. Add Book
2. View All
3. Check Status
4. Update Status
5. Remove Book
6. Exit

Select your option:
```

Menu dispatch behavior:

- `1` calls `func.addBook()`
- `2` calls `func.viewAll()`
- `3` calls `func.checkStatus()`
- `4` calls `func.updateStatus()`
- `5` calls `func.removeBook()`
- `6` sets `run = false`

Functions.java

src/Functions.java

`Functions` is the orchestration layer for book lifecycle operations. It owns the in-memory inventory, performs ISBN lookups, maps domain objects into JSON, and persists the full catalog to `data.json`.

Properties

Property	Type	Description
`bookList`	`JSONArray`	In-memory inventory loaded from and written to `data.json`
`FILE_PATH`	`String`	Relative storage path for the catalog file
`scanner`	`Scanner`	Reads all interactive book prompts from standard input

Constructor dependencies

Type	Description
`Scanner`	Reads console input for add, check, update, and remove workflows

Public methods

Method	Description
`addBook`	Prompts for book details, creates a `PhysicalBook`, serializes it to JSON, appends it to `bookList`, and saves the file
`viewAll`	Prints the full inventory with 1-based numbering and human-readable status labels
`checkStatus`	Looks up a book by exact ISBN match and prints its current availability
`updateStatus`	Looks up a book by exact ISBN match, toggles `available`, saves the file, and prints the new status
`removeBook`	Looks up a book by exact ISBN match, removes it from `bookList`, saves the file, and confirms permanent deletion

Persistence support methods

Method	Description
`loadData`	Loads `data.json` into `bookList`, falls back to an empty array when the file is missing or empty, and resets to an empty array on load failure
`saveData`	Writes the current `bookList` back to `data.json` using `toString(4)` formatting

JSON record shape

Each stored inventory entry is a flat JSON object with these keys:

```
{
  "title": "The Hobbit",
  "author": "J. R. R. Tolkien",
  "isbn": "9780547928227",
  "available": true
}
```

Book.java

src/Book.java

`Book` is the abstract domain base class for inventory records. It provides the shared book fields and encapsulated getters and setters used by `PhysicalBook` and by the JSON mapping in `Functions`.

Properties

Property	Type	Description
`title`	`String`	Book title persisted to JSON and displayed in inventory output
`author`	`String`	Book author persisted to JSON and displayed in inventory output
`isbn`	`String`	Primary lookup key used by `checkStatus`, `updateStatus`, and `removeBook`
`available`	`boolean`	Checkout state used to derive human-readable status text

Public methods

Method	Description
`Book`	Empty base constructor used by subclasses
`getTitle`	Returns `title`
`setTitle`	Assigns `title`
`getAuthor`	Returns `author`
`setAuthor`	Assigns `author`
`getIsbn`	Returns `isbn`
`setIsbn`	Assigns `isbn`
`getAvailable`	Returns `available`
`setAvailable`	Assigns `available`

PhysicalBook.java

src/PhysicalBook.java

`PhysicalBook` is the concrete book type created by `addBook`. It does not add new fields; it simply populates the inherited `Book` state so the object can be serialized into a JSON object.

Inherited properties

Property	Type	Description
`title`	`String`	Assigned directly in the constructor
`author`	`String`	Assigned directly in the constructor
`isbn`	`String`	Assigned directly in the constructor
`available`	`boolean`	Assigned directly in the constructor

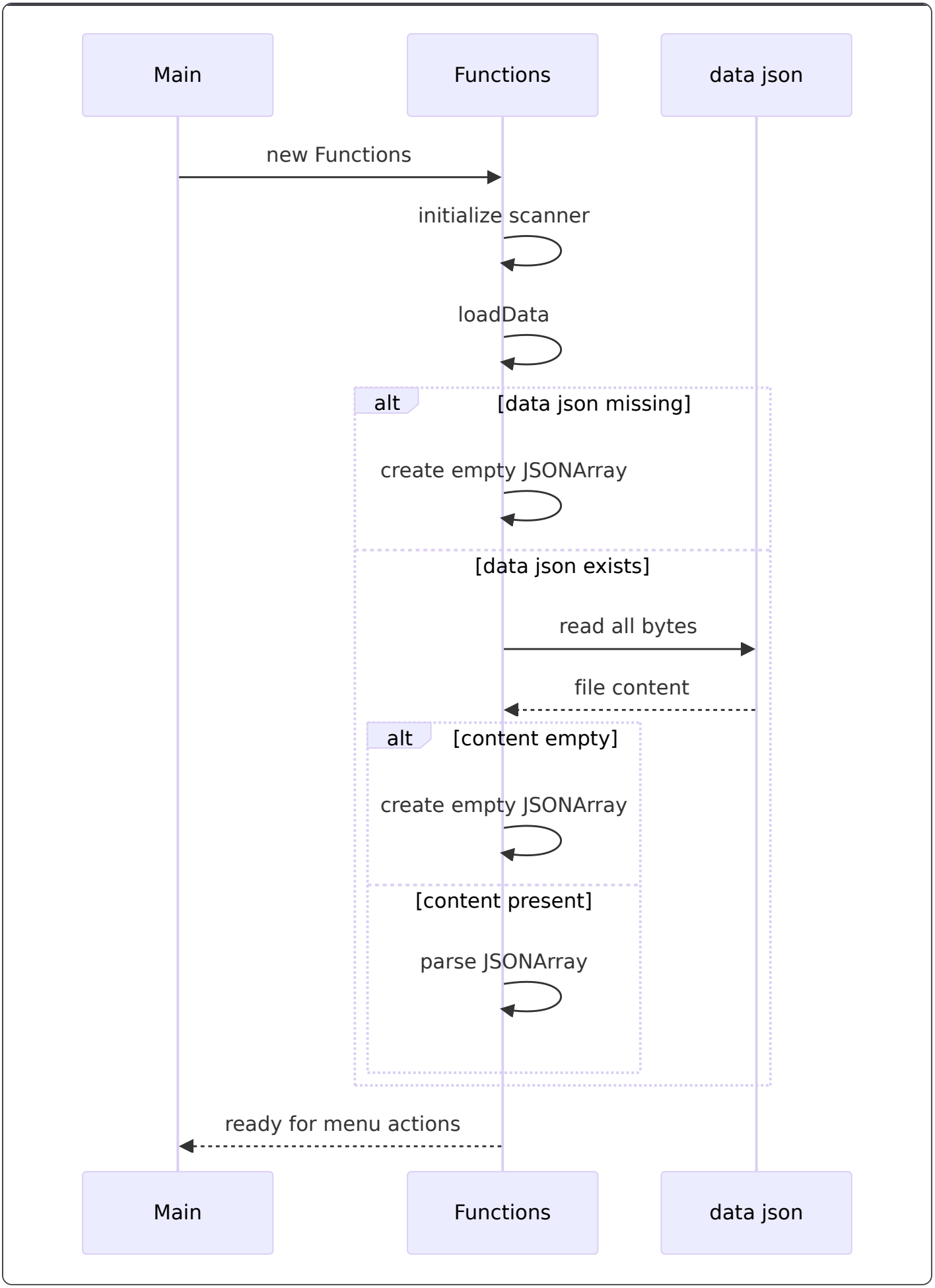
Construction

Constructor input	Type	Effect
`title`	`String`	Stored in inherited `title`
`author`	`String`	Stored in inherited `author`
`isbn`	`String`	Stored in inherited `isbn`
`available`	`boolean`	Stored in inherited `available`

Feature Flows

Application Startup and Data Load

When `Main` starts, it instantiates `Functions`, which immediately creates a `Scanner` and calls `loadData()` before any menu action runs.



Workflow details:

1. `Files.exists(Paths.get(FILE_PATH))` checks for `data.json`.
2. If the file is missing, `bookList` becomes a new empty `JSONArray`.
3. If the file exists, `Files.readAllBytes(...)` loads the full file content.
4. Empty trimmed content becomes a new empty `JSONArray`.
5. Non-empty content is parsed into `bookList`.
6. Any exception prints `[ERROR] Failed to load database: ...` and resets `bookList` to an empty array.

Add Book

`addBook` is the record creation workflow. It collects console input, instantiates `PhysicalBook`, converts that object into a `JSONObject`, appends it to the in-memory array, and writes the entire catalog back to disk.

Exact console prompts:

```
--- Add a New Book ---
Enter Title:
Enter Author:
Enter ISBN:
Is it available? (true/false):
```

Workflow details:

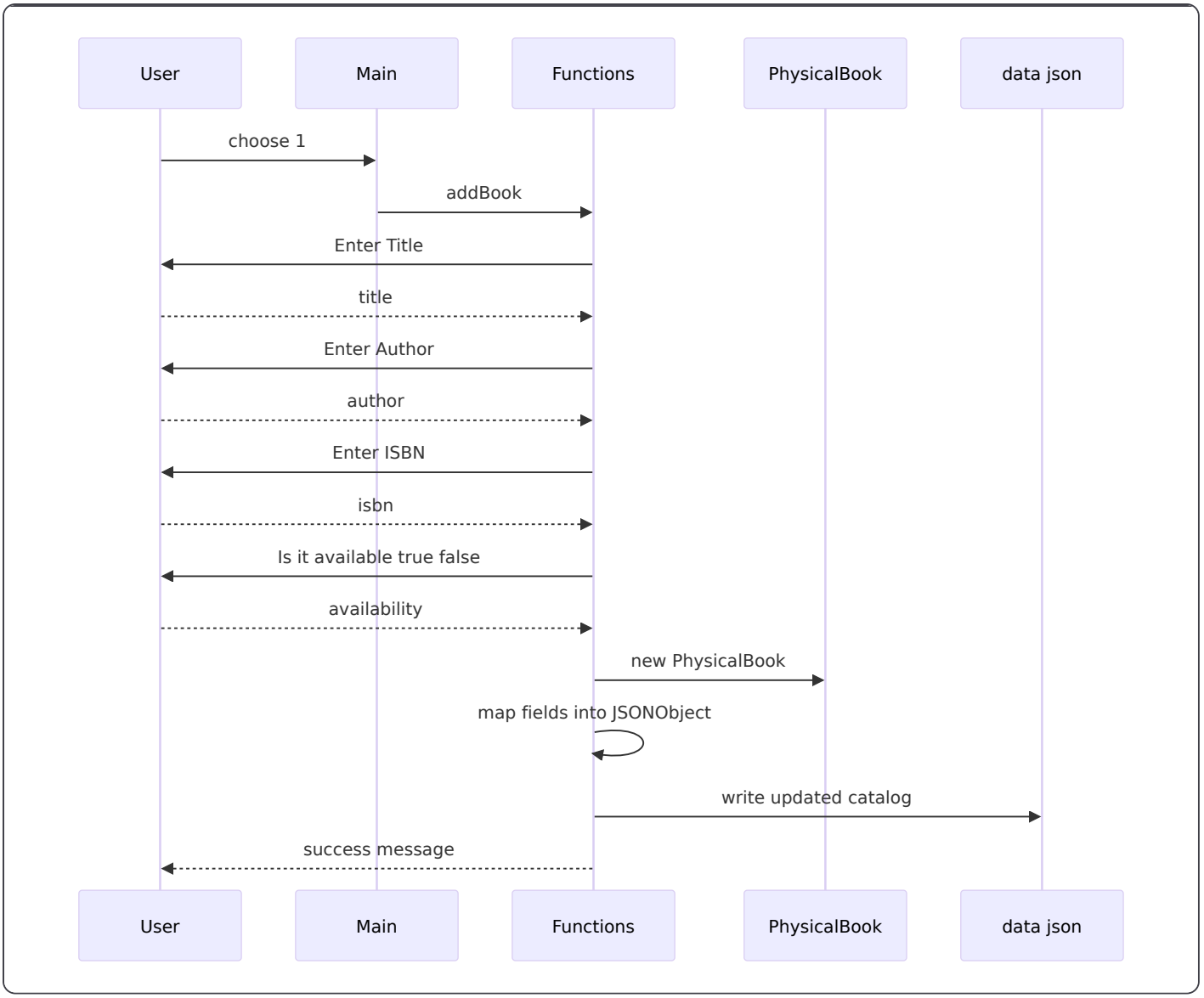
- `title`, `author`, and `isbn` are read with `scanner.nextLine()`.
- Availability is parsed with `Boolean.parseBoolean(...)`.
- A `PhysicalBook` is created with the entered values.
- A new `JSONObject` is built field by field:
  - `title` from `newBook.getTitle()`
  - `author` from `newBook.getAuthor()`
  - `isbn` from `newBook.getIsbn()`
  - `available` from `newBook.getAvailable()`
- The JSON object is appended to `bookList`.
- `saveData()` rewrites `data.json`.
- Success output is:

```
[SUCCESS] Book '...title...' has been added to the system.
```

Error path:

- A `JSONException` during JSON construction prints:

```
[ERROR] Failed to process book data: ...
```



View All

`viewAll` prints the complete catalog in its current in-memory order. It derives a readable status string from each record's `available` flag.

Exact console text:

```
--- Library Inventory ---
```

Empty inventory behavior:

```
The library is currently empty.Functions both
```

Workflow details:

- If `bookList.length() == 0`, the method prints the empty message and returns immediately.
- Otherwise, it iterates from `0` to `bookList.length() - 1`.
- Each row is printed with a 1-based index.
- The status label is derived as:

- `true` → `Available`
- `false` → `Checked Out`

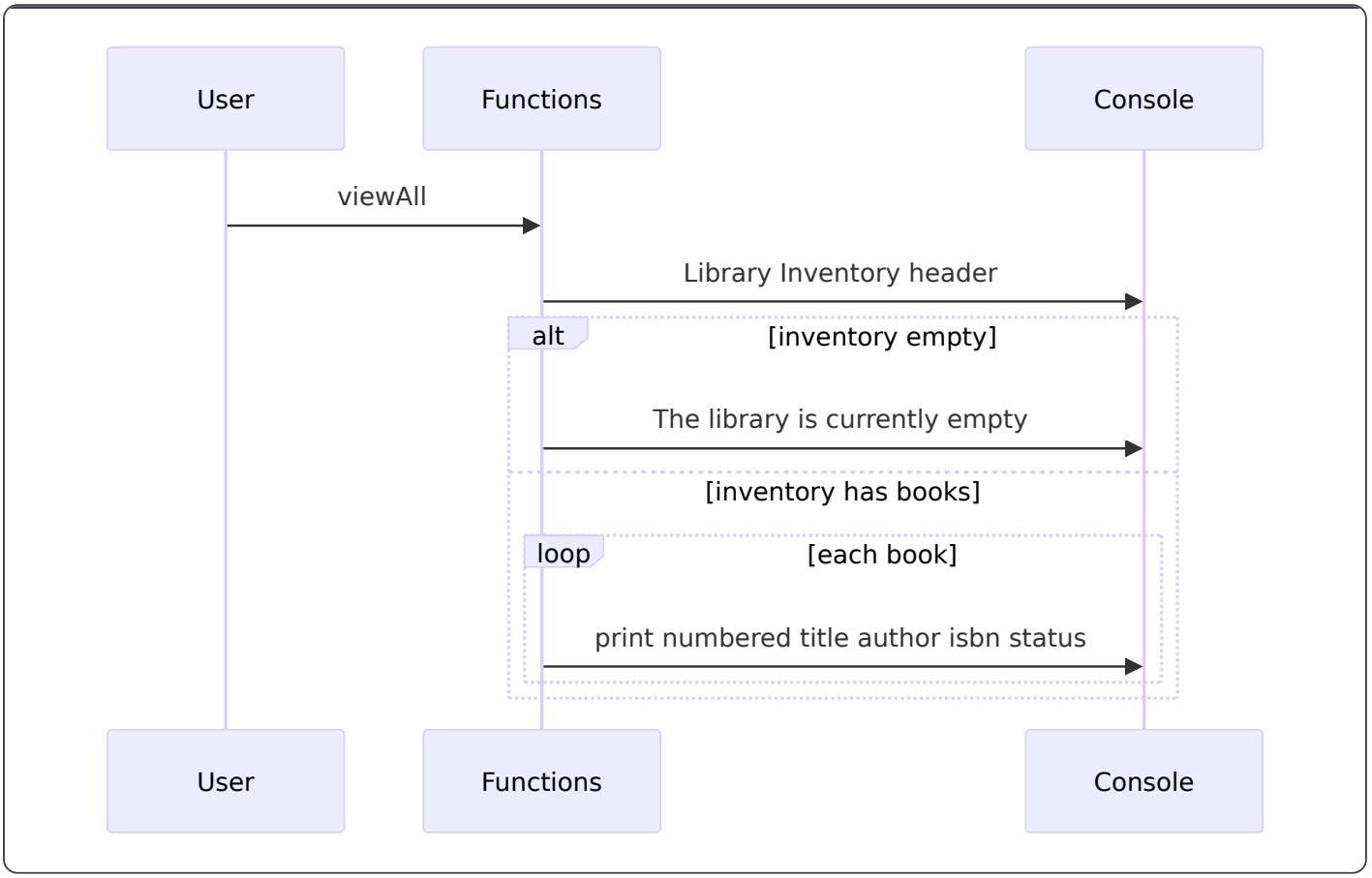
Output line shape:

```
1. Title: ... | Author: ... | ISBN: ... | Status: Available
```

Error path:

- A `JSONException` while reading any entry prints:

```
[ERROR] Corrupted data found while reading the list.
```



Check Status

`checkStatus` performs an exact ISBN lookup and prints the availability of the first matching record.

Exact console prompt:

```
Enter the ISBN of the book to check:
```

Lookup behavior:

- Reads the search value with `scanner.nextLine()`.
- Compares each stored record with exact string equality:

    - `book.getString("isbn").equals(searchIsbn)`
- The first match wins.
- On match, the method prints the book title and readable status, then returns immediately.

Success output shape:

```
Status for '...title...': Available
```

or

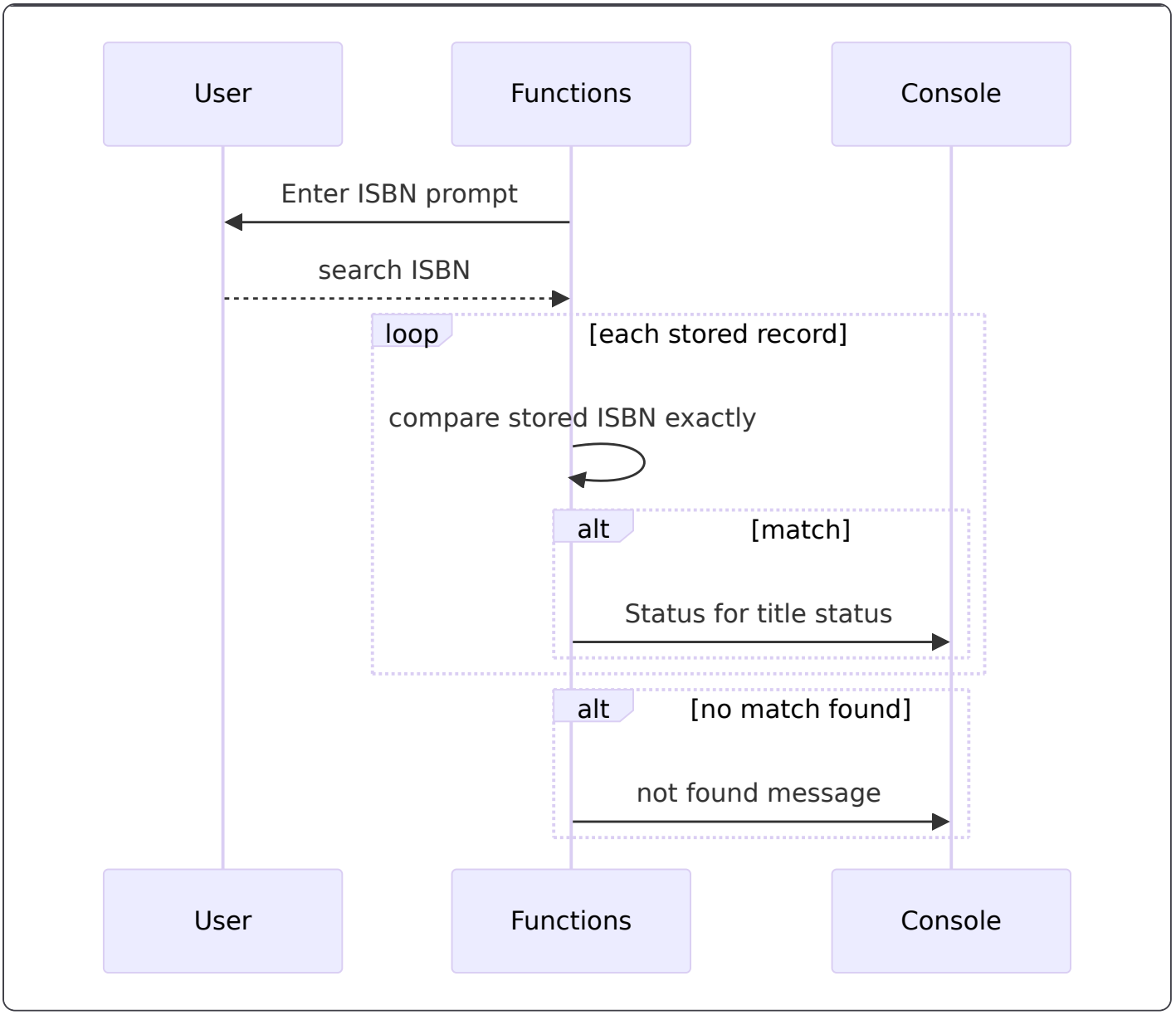
```
Status for '...title...': Checked Out
```

Not found output:

```
[NOT FOUND] No book matches the ISBN: ...
```

Error path:

```
[ERROR] Failed to read book status.
```



Update Status

`updateStatus` performs an exact ISBN lookup, flips the `available` flag, persists the change, and prints the new human-readable status.

Exact console prompt:

```
Enter the ISBN of the book to update:
```

Workflow details:

- Reads the ISBN with `scanner.nextLine()`.
- Searches `bookList` using exact string equality on the stored `isbn`.
- When a match is found:
  - `currentStatus` is read from `book.getBoolean("available")`
  - the stored value is replaced with `!currentStatus`
  - `saveData()` persists the updated array
  - the success message uses the toggled value to derive the readable label

Readable status derivation after toggle:

- `true → Available`
- `false → Checked Out`

Success output shape:

```
[SUCCESS] The status for '...title...' is now: Available
```

or

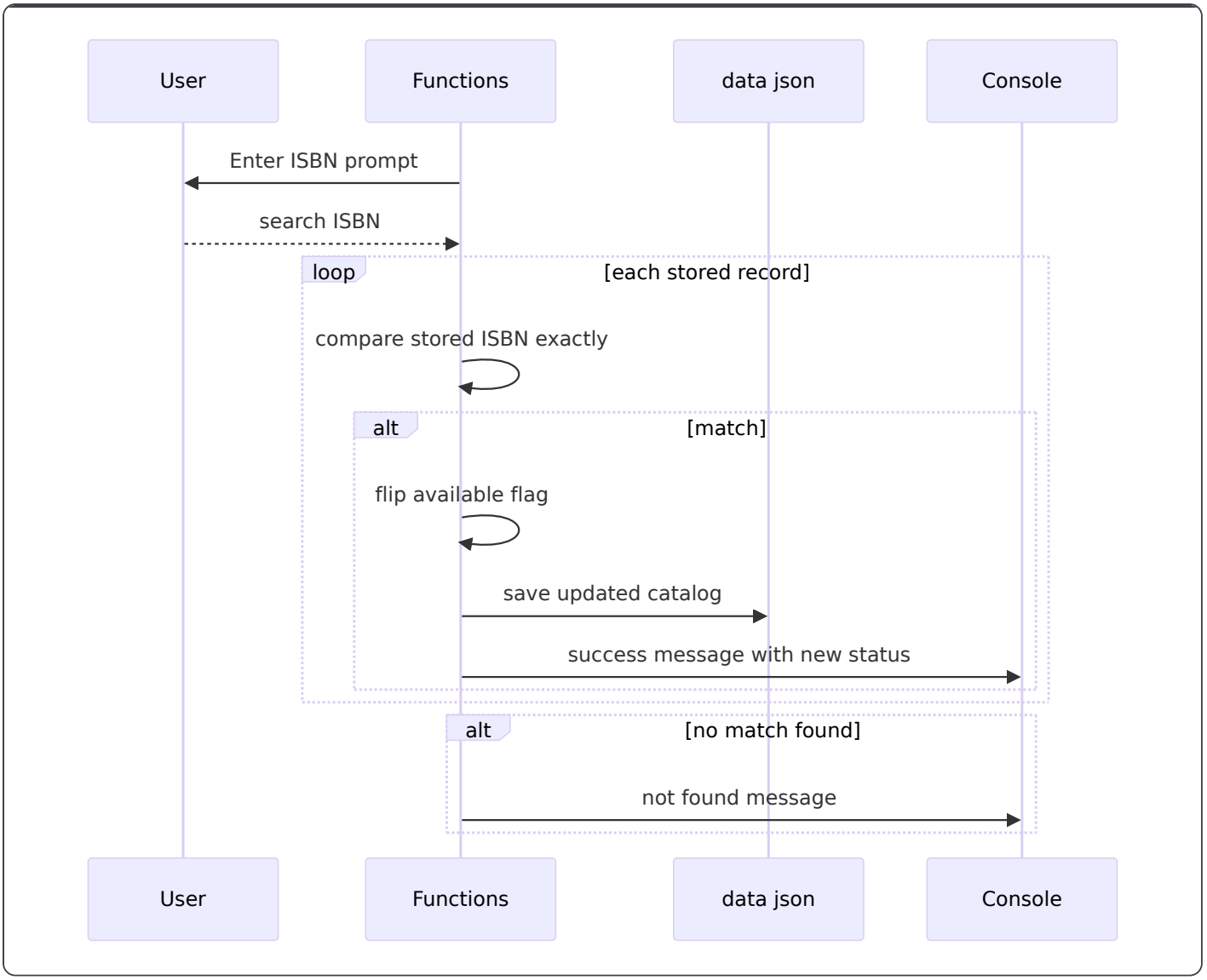
```
[SUCCESS] The status for '...title...' is now: Checked Out
```

Not found output:

```
[NOT FOUND] No book matches the ISBN: ...
```

Error path:

```
[ERROR] Failed to update book status.
```



Remove Book

`removeBook` performs an exact ISBN lookup and permanently removes the first matching record from `bookList`.

Exact console prompt:

```
Enter the ISBN of the book to remove:
```

Workflow details:

- Reads the ISBN with `scanner.nextLine()`.
- Searches `bookList` with exact string equality on `isbn`.
- When a match is found:
  - captures the book `title`
  - removes the array element with `bookList.remove(i)`
  - calls `saveData()`
  - prints the permanent removal confirmation
- The first matching record is removed and the method returns immediately.

Success output:

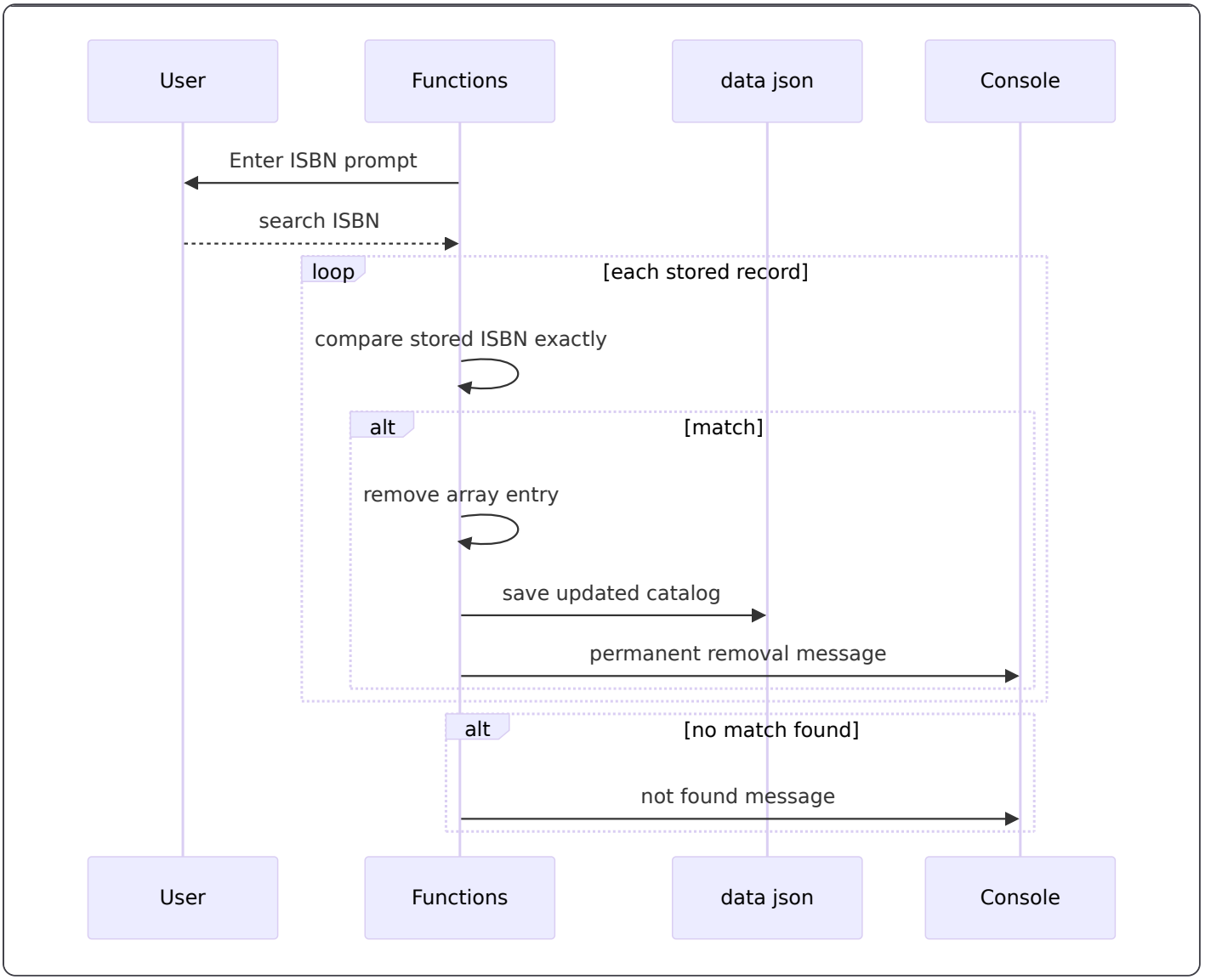
```
[SUCCESS] The book '...title...' has been permanently removed.
```

Not found output:

```
[NOT FOUND] No book matches the ISBN: ...
```

Error path:

```
[ERROR] Failed to remove the book.
```



Readable Status Mapping

The human-readable status shown in `viewAll`, `checkStatus`, and `updateStatus` is derived from the boolean `available` field.

available	Display text
true	Available
false	Checked Out

This mapping is applied directly from the JSON record. No additional state is stored for status labels.

State Management

- `bookList` is the single in-memory source of truth for the catalog.
- `data.json` mirrors the current `bookList` state after each mutation.
- `loadData()` hydrates the full inventory once during `Functions` construction.
- `saveData()` rewrites the complete array with `bookList.toString(4)`.
- ISBN-based operations use exact string equality on the stored `isbn` field.
- The displayed inventory order is the current array order, with `viewAll` numbering records starting at 1.
- `available` is the only operational status flag and is flipped in place by `updateStatus`.

Error Handling

Method	Failure condition	Console output
`loadData`	File read, parse, or general load failure	`[ERROR] Failed to load database: ...`
`saveData`	File write failure	`[ERROR] Failed to save database: ...`
`addBook`	JSON object construction failure	`[ERROR] Failed to process book data: ...`
`viewAll`	JSON corruption while reading entries	`[ERROR] Corrupted data found while reading the list.`
`checkStatus`	JSON read failure during lookup	`[ERROR] Failed to read book status.`
`updateStatus`	JSON read or update failure	`[ERROR] Failed to update book status.`
`removeBook`	JSON read or removal failure	`[ERROR] Failed to remove the book.`

Additional runtime behavior:

- `loadData()` falls back to an empty `JSONArray` when `data.json` is missing or blank.

- `checkStatus`, `updateStatus`, and `removeBook` all exit immediately after the first matching ISBN.
- `saveData()` handles its own exception and does not rethrow, so the caller continues after the attempt to write the file.

Dependencies

Dependency	Used by	Purpose
`java.util.Scanner`	`Main`, `Functions`	Console input for menu selection and book prompts
`java.nio.file.Files`	`Functions`	File existence checks and file reads
`java.nio.file.Paths`	`Functions`	Builds the `data.json` path
`java.io.FileWriter`	`Functions`	Writes the full catalog back to disk
`org.json.JSONArray`	`Functions`	In-memory representation of the full inventory
`org.json.JSONObject`	`Functions`	Per-book JSON serialization and mutation
`org.json.JSONException`	`Functions`	Handles JSON parsing and access failures
`Book`	`PhysicalBook`	Abstract domain base class
`PhysicalBook`	`Functions.addBook`	Concrete book object created before JSON serialization
`data.json`	`Functions`	Persistent local inventory store

Key Classes Reference

Class	Responsibility
`Main.java`	Starts the console menu and dispatches menu choices into `Functions`
`Functions.java`	Orchestrates add, view, lookup, update, remove, and JSON persistence workflows
`Book.java`	Defines the shared book fields and accessors
`PhysicalBook.java`	Provides the concrete book instance used during record creation